Dipartimento di Elettronica e Informazione

Politecnico di Milano

prof. ***Giovanni Denaro***
prof. ***Carlo Ghezzi***
prof. ***Angelo Morzenti***
prof. ***Pierluigi San Pietro***

20133 Milano (Italia)
Piazza Leonardo da Vinci, 32
Tel. (39) 02-2399.3400
Fax (39) 02-2399.3411

II Prova di Ingegneria del Software

1 Luglio 2002

Cognome

Nome

Matricola

Sezione (segnarne una) Denaro ☐, Ghezzi ☐, Morzenti ☐, SanPietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno presi in considerazione.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.30m.
6. Punteggio totale a disposizione: 40 punti nominali (10 per ogni esercizio), corrispondenti a 13/30.

Esercizio 1:
Sono date le seguenti definizioni di una classe Stack e di una classe BoundedStack

```
public class Stack {
//OVERVIEW: una pila di  elementi di tipo Object
  public Stack( )
        //EFFECTS: inizializza this alla pila vuota
  public void push(Object v)
        //EFFECTS: inserisce v in cima a this
  public Object topPop( ) throws EmptyStackException
        //REQUIRES: this.size()>0
        //EFFECTS: restituisce e elimina cima di this
public int size( )
        // EFFECTS: restituisce il numero di elementi di this
}

public class BoundedStack extends Stack {
//OVERVIEW: una pila di  elementi di tipo Object di dimensioni massime pari a cento
public BoundedStack()
        //EFFECTS: inizializza this alla pila vuota
public void push(Object v)
        //REQUIRES:  0<this.size()<=100
        //EFFECTS: inserisce v in cima alla pila
}
```

Si supponga che il rep della classe Stack sia dichiarato private.
La soluzione proposta verifica il principio di sostituzione di Liskov?
La risposta deve essere motivata.

Soluzione: La soluzione non verifica il principio di sostituzione perché la precondizione di BoundedStack.push (this.size()<100) e' piu' forte di quella di Stack.push (true), violando la regola dei metodi.

Esercizio 2

Si consideri la classe Mystery riportata di seguito.
Si modifichi la classe in modo che gli oggetti che essa genera possano essere usati da più thread concorrenti, garantendo che:

1. i metodi *bingo* e *bango* vengano eseguiti da ciascun thread in mutua esclusione;
2. un thread che esegue *bingo* e trova che la condizione $n>m$ non è vera viene sospeso fino al momento in cui la condizione diventa vera a causa dell'esecuzione di *bango* da parte di un altro thread,
3. analogamente, un thread che esegue *bango* e trova che la condizione $m>1$ non è vera viene sospeso fino al momento in cui la condizione diventa vera a causa dell'esecuzione di *bingo* da parte di un altro thread.

```
class Mistery {
        private int n=10;  private int m=1;
        …
        public bingo {
                if (n>m) {
                        n--; m++;
                        esegui operazione 1;
                }
        }
        public bango {
                if (m>1) {
                        n++; m--;
                        esegui operazione 2;
                }
        }
}
```

Soluzione

```
public class Mistery {
        .........
        private int n=10,
        private int m=1;
        public synchronized void bingo {
                while  ( !(n>m) )
                        try  { wait(); }
                        catch(InterruptedException e) {}
                n--;
                m++;
                esegui operazione 1;
                notifyAll();
        }

        public synchronized void bango {
                while ( !( m>1) )
                        try  { wait(); }
                        catch(InterruptedException e) {}
                n++;
                m--;
                esegui operazione 2;
                notifyAll();
        }
}
```

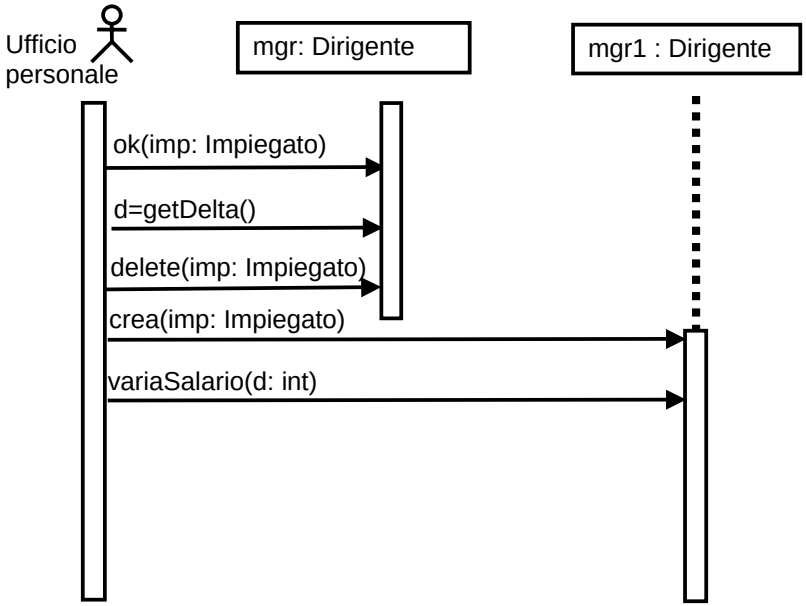

Esercizio 3 (parte a)
Si consideri la fase di analisi e specifica dei requisiti di un nuovo software che deve essere realizzato e il seguente frammento di descrizione del dominio applicativo (la struttura di una certa azienda):

"Le divisioni aziendali hanno almeno 1 dipendente, ma il numero massimo non è predeterminato. Ogni dipendente ha un codice fiscale, un nome, un cognome e un salario; deve essere possibile modificare il salario mediante un'operazione variaSalario(x), dove x è un parametro intero che rappresenta la variazione di salario. Un dipendente è assegnato a una e una sola divisione. I dipendenti possono essere impiegati o dirigenti. Un dirigente coordina al massimo 20 impiegati e fornisce due operazioni, add e delete, con i quali si può aggiungere o eliminare un impiegato dall'insieme degli impiegati coordinati dal dirigente. Un impiegato è coordinato da uno o anche da due dirigenti."

1. **Rappresentare questa descrizione mediante un class diagram di UML.**

2. **Che cosa cambierebbe nel diagramma se si cambiasse il vincolo "un dirigente coordina al massimo 20 impiegati" in "un dirigente coordina da 5 a 20 impiegati"?**

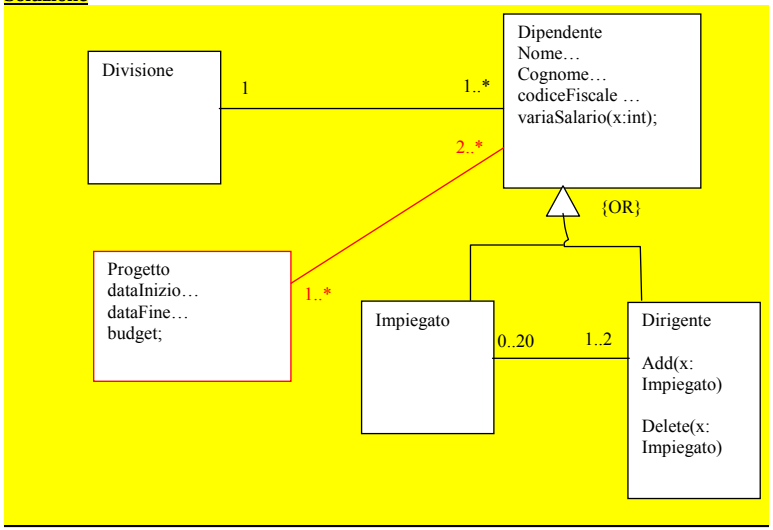
3. **Che cosa cambierebbe nel diagramma se si aggiungesse questo frammento di specifica: "ogni dipendente lavora in almeno un progetto e in ciascun progetto lavorano almeno 2 dipendenti. Ogni progetto ha una data di inizio, una data di fine e un budget."**



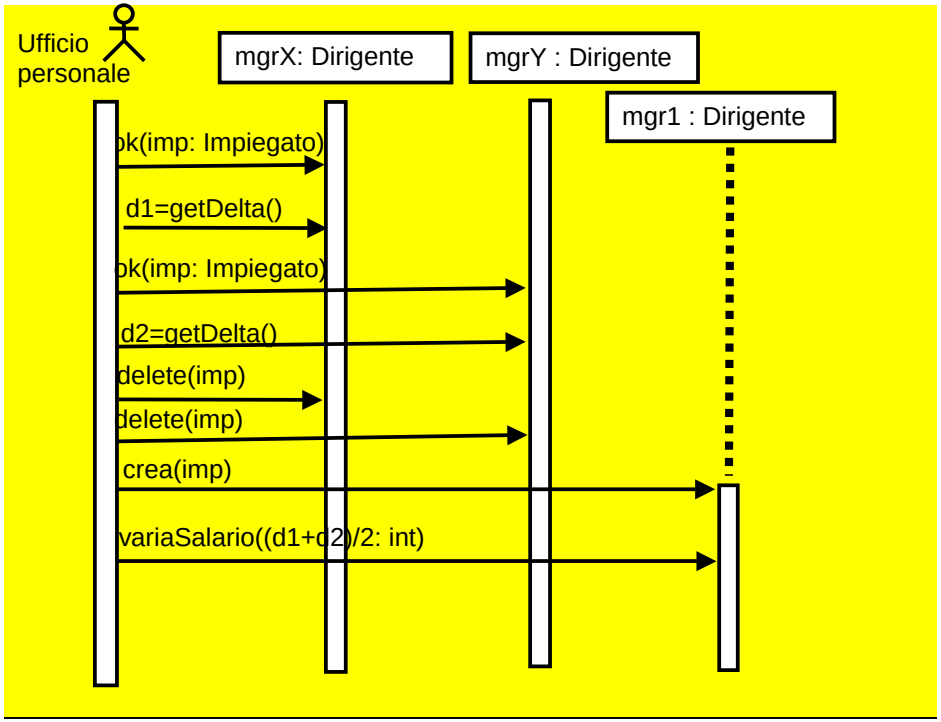
Esercizio 3 (parte b)
Si consideri il sequence diagram riportato in figura, che descrive la promozione di un impiegato a dirigente da parte dell'ufficio personale. Il diagramma descrive il fatto che l'uffico personale deve avere l'approvazione del dirigente da cui l'impiegato dipende e da questo deve farsi dare l'incremento di stipendio da applicare all'impiegato nella promozione a dirigente. (Si noti che il diagramma descrive solo lo scenario in cui il dirigente dà l'approvazione).

4. **Si definisca un nuovo sequence diagram che descrive il caso in cui il dipendente è coordinato da due dirigenti. Se entrambi danno l'approvazione, l'incremento di stipendio è il valor medio delle due proposte di incremento.**

Soluzione



1. vedi diagramma delle classi qui sopra
2. sostituire 0..20 con 5..20
3. aggiungere la parte in rosso
4: vedi sequence diagram qui sotto



Esercizio 4 (parte a)
Si consideri la specifica del seguente metodo:

```
static int triangolo (int a, int b, int c) {
        // REQUIRES: a>0 and b>0 and c>0
        // EFFECTS: restituisce 1 se i tre lati definiscono un triangolo equilatero,
        // 2 se definiscono un triangolo isoscele,
        // 3 se definiscono un triangolo scaleno,
        // 0 se a, b, e c non possono essere lati di un triangolo
}
```

Si definisca un insieme di dati di test in base a una strategia di tipo "black box" (cioè funzionale), motivando sinteticamente ciascuna scelta.

Esercizio 4 (parte b) Si consideri la seguente implementazione (che non tiene conto dell'ultima parte della clausola EFFECTS) del metodo precedente:

```
static int triangolo (int a, int b, int c) {
        if ((a==b) && (b==c)) return 1;
        if ((a==b) || (b==c) || (a==c)) return 2;
        else return 3;
}
```

Si consideri la seguente formulazione di una strategia di tipo "white box"(cioè strutturale): deve esistere, per ciascun cammino percorribile del flusso di controllo, uno e un solo dato di test che ne genera la copertura.

1. **Quanti casi di test sono necessari? _____**
2. **Si descrivano i test scelti:**

Esercizio 4 (parte c) Si consideri la seguente implementazione alternativa della funzione precedente:

```
static int triangolo (int a, int b, int c) {
        if ((a==b) && (b==c)) return 1;
        else if (a==b) return 2;
        else if (b==c) return 2;
        else if (a==c) return 2;
        else return 3;
}
```

Considerando sempre la precedente formulazione del criterio di copertura dei cammini:

1. **Quanti casi di test sono necessari per questo programma? _____**
2. **Si descrivano i test scelti:**

Soluzione a:
I dati <5, 5, 5>, <5, 7, 7>, <5, 6, 7> e <5, 6, 12> corrispondono alle quattro parti della clausola EFFECTS e per essi il metodo deve restituire, rispettivamente, i valori 1, 2, 3 e 0.
Soluzione b:
Servono 3 dati di test. I dati possono essere <5, 5, 5>, <5, 5, 7>, <5, 6, 7>.
Soluzione c:
Servono 5 dati di test. I dati possono essere <5, 5, 5>, <5, 5, 7>, <5, 7, 7>, <5, 7, 5>, <5, 6, 7>.